A MIDTERM PROGRESS REPORT

On

BLOCKCHAIN SECURE VAULT

Submitted in partial fulfillment of the requirements for the award of

the degree of

BACHELOR OF TECHNOLOGY

Computer Science and Engineering

SUBMITTED BY

EKAMPREET SINGH	GAURAV PRASHAR	GAURAV SHARMA
2302515	2302518	2302519

UNDER THE GUIDANCE OF

ASST PROF. MEETALI

OCTOBER 2026

Department of Computer Science and Engineering

GURU NANAK DEV ENGINEERING COLLEGE,

LUDHIANA

Contents

List of Tables **4**

List of Figures **5**

1 Introduction **6**
1.1 Brief introduction about the project to be developed 6
1.2 Technology Used 6
1.3 Field of Project 6
1.4 Special Technical Terms 7
1.5 Objectives 7

2 System Requirements **8**
2.1 Software Requirements 8
2.2 Hardware Requirements 8
2.3 Mobile Application Requirements 9
2.4 Requirements Related to Project Objectives 9

3 Software Requirement Analysis **10**
3.1 Problem Definition 10
3.2 Existing System 10
3.3 Proposed System 10
3.3.1 Secure Mobile Document Management 11
3.3.2 Role-Based Access Control 11
3.3.3 Document Search, Organization and Controlled Viewing 11
3.4 Functional Requirements 11
3.5 Modules of the Proposed System 12
3.5.1 Module 1: Mobile Application Module 12
3.5.2 Module 2: Document Management Module 13
3.5.3 Module 3: Role-Based Access Control Module 13
3.5.4 Module 4: Document Search and Organization Module 13
3.5.5 Module 5: Controlled Document Viewing Module 14
3.6 Non-Functional Requirements 14
3.6.1 Security 14
3.6.2 Usability 14
3.6.3 Reliability 14
3.6.4 Performance 14
3.6.5 Maintainability 14

4 Software Design **15**
4.1 System Architecture 15
4.2 System Use Case Diagram 16
4.3 DFD Level 0 (Context Diagram) 17
4.4 DFD Level 1 18
4.5 Sequence Diagram – Public Dual-Check Verification 19
4.6 Activity Diagram – Controlled Document Access & Decryption 20
4.7 Class / Entity Diagram 20
4.8 Deployment Diagram 21
4.9 Design Summary 22

5 Testing Module **24**
5.1 Testing Objectives 24
5.2 Testing Techniques 24
5.2.1 Functional Testing 24
5.2.2 Integration Testing 25
5.2.3 Access-Control Testing 25
5.2.4 User Interface Testing 25
5.3 Test Cases 26
5.4 Security Testing 26
5.5 Expected Testing Outcome 27

6 Performance of Project Developed So Far **28**
6.1 Mobile Application Performance 28
6.2 Document Management Performance 28
6.3 Role-Based Access Control Performance 29

7 Output Screens **30**

List of Tables

1 Software Requirements 8
2 Hardware Requirements 9
3 Functional Requirements of the Major Project 12
4 Test Cases for the Major Project 26

List of Figures

1 System Architecture of the Secure Document Vault 15
2 System Use Case Diagram 16
3 DFD Level 0 (Context Diagram) 17
4 DFD Level 1 of the Blockchain Secure Vault System 18
5 Sequence Diagram for Public Dual-Check Verification 19
6 Activity Diagram for Controlled Document Access and Decryption 20
7 Class / Entity Diagram of the Secure Document Vault 21
8 Deployment Diagram of the Secure Document Vault 22
9 Secure Document Vault Home Page – View 1 30
10 Secure Document Vault Home Page – View 2 30
11 Secure Document Vault Home Page – View 3 31
12 Secure Document Vault Home Page – View 4 31
13 User Registration Using Email and Password 32
14 User Registration Using Google Authentication 32
15 User Sign In Using Email and Password 33
16 User Registration Using OTP Verification 33
17 User Sign In Using Google Authentication 34
18 Secure Document Vault Home Page After User Authentication 34
19 Secure Document Vault Dashboard 35
20 Document Upload Interface 35
21 Document Verification Interface 36
22 Organization Dashboard 36
23 Organization Registration Interface 37
24 Organization Management Interface 37
25 User Profile Information and Logout Interface 38

1 Introduction

1.1 Brief introduction about the project to be developed

This project is an extended and enhanced version of the previously developed Blockchain Secure Vault, continued as a major project through the development of a dedicated mobile application for secure document management. The existing system provides encrypted document storage, decentralized storage, and blockchain-based integrity verification. The major project focuses on extending this foundation with a convenient mobile interface while improving document organization, accessibility, and access control. The proposed system combines end-to-end encryption, decentralized storage, blockchain-based verification, and application-level security to provide a secure environment for managing sensitive digital documents [1].

The mobile application will enable authorized users to securely access, organize, search, share, and manage their encrypted documents. It will introduce a file-manager interface with folders and subfolders, document search, role-based access control, and temporary access permissions. Users will also be able to view supported documents within the application and access downloaded documents through controlled in-app workflows. Temporary access can be granted for a predefined duration and automatically becomes invalid after expiration, while role-based permissions ensure that users can perform only authorized operations. The mobile application will communicate with the existing backend and blockchain infrastructure, allowing the security mechanisms developed during the minor project to be utilized and extended in the major project.

The project aims to provide a practical and secure mobile solution for document management while maintaining confidentiality, integrity, and controlled accessibility. By combining blockchain, cryptography, decentralized storage, secure backend services, and mobile application development, the project demonstrates the practical application of these technologies in a comprehensive secure document management system.

1.2 Technology Used

- **Mobile Application:** React Native for developing the cross-platform mobile application
- **Backend:** Node.js and Express for REST APIs, authentication, access control, and document management
- **Database:** MongoDB for user accounts, document metadata, folders, permissions, and access records
- **Blockchain:** Solidity smart contracts with Hardhat and Ethers.js for on-chain integrity verification
- **Storage:** IPFS with Pinata for decentralized storage of encrypted documents
- **Cryptography:** AES-256 for document encryption and SHA-256 for integrity hashing [2]
- **Document Processing:** PDF.js, Mammoth.js, and suitable mobile document-rendering libraries for secure in-app document viewing

1.3 Field of Project

- Information and Cyber Security
- Blockchain and Decentralized Storage
- Secure Mobile Application Development

- Secure Document and File Management
- Access Control and Privacy

1.4 Special Technical Terms

- **End-to-End Encryption (E2EE):** Documents are encrypted before storage and decrypted only when accessed by an authorized user.
- **Content Hash:** A cryptographic fingerprint used to identify a document and detect unauthorized modification.
- **On-chain Verification:** Verification of document integrity by comparing its computed hash with the corresponding blockchain record.
- **Role-Based Access Control (RBAC):** A security mechanism in which document permissions are assigned according to the user's role and authorized operations.
- **Temporary Access:** Time-limited permission that automatically expires after a predefined duration.
- **Immutable Ledger:** A tamper-resistant blockchain record used to maintain trustworthy document integrity information.
- **In-App Document Viewing:** A controlled mechanism for viewing supported documents within the mobile application without unnecessarily exposing decrypted files to external applications.

1.5 Objectives

1. To develop a secure mobile application for streamlined document management and organization.
2. To establish role-based access control for secure document sharing.
3. To enable efficient document search and folder-based organization with in-app viewing through secure access mechanisms

2 System Requirements

The Major Project extends the existing Secure Document Vault by introducing a mobile application for secure document management and controlled access. The system requirements therefore focus primarily on the hardware and software required to develop and operate the mobile application and its supporting services.

2.1 Software Requirements

The following software technologies and development tools are required for the development and execution of the extended system:

Table 1: Software Requirements

Software / Technology	**Purpose**
React Native	Development of the cross-platform mobile application
Expo	Development, testing, and execution of the React Native mobile application
Expo Router	Management of application navigation and screen routing
TypeScript	Type-safe development of the mobile application
NativeWind	Styling and responsive user-interface development
Axios	Communication between the mobile application and backend REST APIs
Expo SecureStore	Secure storage of authentication tokens and sensitive session information on the mobile device
Node.js	Runtime environment for the existing backend services
Express.js	Backend REST API framework used by the mobile application
MongoDB	Storage of document-related metadata, user information, sharing information, and access-related data
Git / GitHub	Source-code management and version control
Visual Studio Code	Primary development environment for source-code development and debugging

The mobile application communicates with the existing backend through REST APIs rather than implementing a separate backend. This allows the mobile application to reuse the existing document-management and authorization services.

The system provides authentication and authorization mechanisms that support controlled access to documents. These mechanisms include existing and modified JWT-based authentication and role-based authorization functionalities, adapted to meet the requirements of the enhanced system.

2.2 Hardware Requirements

The following hardware is required for development, testing, and operation of the system:

Table 2: Hardware Requirements

Hardware	Minimum / Recommended Requirement
Development Computer	Intel Core i5 or equivalent processor
RAM	Minimum 8 GB; 16 GB recommended for smoother development
Storage	Minimum 20 GB of available storage for development tools, dependencies, Android SDK, and project files
Internet Connection	Required for package installation, backend communication, API access, and development services
Android Device / Emulator	Android device or emulator for testing the mobile application
Mobile Storage	Sufficient storage for application installation and temporary document handling

2.3 Mobile Application Requirements

The mobile application requires an Android-compatible environment for installation and execution. During development, the application can be tested using either a physical Android device or an Android emulator.

The application requires network connectivity to communicate with the existing backend services. Authentication information is securely stored on the mobile device, while document-related operations are performed through the backend APIs.

2.4 Requirements Related to Project Objectives

The system requirements directly support the three major objectives of the project:

1. **Secure Mobile Document Management**

   The React Native and Expo environment provides the platform for developing the mobile application through which users can manage their documents.

2. **Role-Based Access Control**

   The existing authentication and authorization infrastructure provides role and organization-level access checks, which are used to control document-related operations.

3. **Document Search, Organization and Controlled Viewing**

   The backend provides document retrieval, sharing, downloading, and viewing operations that can be accessed by the mobile application. File and folder organization is being extended as part of the Major Project.

The requirements described above are sufficient to support the implementation and testing of the major-project objectives without unnecessarily repeating the complete requirements of the original Minor Project.

3 Software Requirement Analysis

3.1 Problem Definition

The existing Secure Document Vault provides a secure foundation for storing, accessing, sharing, and verifying digital documents. However, the earlier system primarily focused on the underlying secure document infrastructure and web-based access. There is a need to extend this functionality to a mobile environment so that users can manage their documents more conveniently while maintaining controlled access.

The Major Project addresses this requirement by extending the existing system with a mobile application focused on secure document management, controlled sharing, document retrieval, and organization. The mobile application communicates with the existing backend services and provides a mobile-oriented interface for the required document operations. The repository contains both an Expo/React Native mobile application and a shared Node.js/Express backend.

The main problems addressed by the Major Project are:

- Difficulty in managing and accessing documents conveniently through a mobile platform.
- Requirement for controlled document sharing based on user roles and permissions.
- Difficulty in efficiently locating and organizing documents as the number of stored documents increases.
- Requirement for controlled access to documents instead of unrestricted viewing or sharing.
- Need to extend the existing Secure Document Vault without duplicating the complete backend system.

Therefore, the Major Project concentrates on providing a mobile interface and supporting functionality for secure document management, role-based access, document search, folder-based organization, and controlled document viewing.

3.2 Existing System

The existing system was developed as the Minor Project and provides the foundation for the current Major Project. It includes secure document management functionality together with authentication, document storage, sharing and verification.

For the Major Project, these existing services are reused wherever possible. The mobile application communicates with the existing backend through APIs instead of creating an independent backend. This reduces duplication and allows the new mobile functionality to work with the existing document-management infrastructure.

3.3 Proposed System

The proposed system is an extension of the Secure Document Vault that introduces and enhances mobile-based document management. The system provides users with a mobile interface through which they can manage documents while access to documents is controlled according to authorization rules.

The proposed system focuses on three major areas:

3.3.1 Secure Mobile Document Management

A mobile application is developed using React Native and Expo to provide users with a convenient interface for document-related operations. The application communicates with the existing backend through REST APIs.

The mobile application provides the interface required for managing documents and accessing the functionality of the Secure Document Vault from a mobile device. The repository identifies the mobile application as an Expo/React Native client located under the mobile application directory.

3.3.2 Role-Based Access Control

The proposed system provides controlled access to documents according to user roles and organizational permissions. The authorization layer validates authenticated users and applies organization or membership-based access checks before allowing protected operations.

The existing implementation includes JWT-based authentication. Role-based access control is identified as an implemented functional requirement in the project analysis.

This mechanism forms the basis for secure document sharing by ensuring that users can perform document operations only when they have the required permissions.

3.3.3 Document Search, Organization and Controlled Viewing

The system provides document retrieval functionality so that users can locate and access documents associated with their authorized user or organization context. Document download and viewing functionality is also present in the existing system.

The Major Project further focuses on improving document organization through folder-based management and providing controlled in-app access to documents. File/folder management is currently not implemented, making it one of the areas to be further developed during the project.

3.4 Functional Requirements

The functional requirements relevant to the Major Project are as follows:

Table 3: Functional Requirements of the Major Project

ID	Functional Requirement	Description
FR-01	Mobile Application	Provide a mobile interface for secure document-management operations.
FR-02	User Authentication	Allow authenticated users to access protected mobile functionality.
FR-03	Document Management	Allow users to access and manage documents through the mobile application.
FR-04	Document Search	Allow users to retrieve documents according to their authorized user or organization context.
FR-05	Document Sharing	Allow documents to be shared with authorized users.
FR-06	Role-Based Access Control	Restrict document operations according to user roles and permissions.
FR-07	Controlled Access	Ensure protected document operations are available only to authorized users.
FR-08	Document Viewing	Allow authorized users to view documents through the application.
FR-09	Folder/File Organization	Organize documents using file/folder-based management.
FR-10	Mobile-to-Backend Communication	Communicate with the existing backend through REST APIs.

The project analysis specifically identifies document search, document download, document viewing, role-based access control, temporary access, and the mobile application among the implemented requirements, while file/folder management is not implemented.

3.5 Modules of the Proposed System

To keep the Major Project focused on its defined objectives, the relevant modules are organized as follows.

3.5.1 Module 1: Mobile Application Module

Purpose:

Provide a mobile interface for accessing and managing documents.

Main responsibilities:

- Provide mobile screens for document-related operations.
- Communicate with backend APIs.
- Manage authenticated user sessions.
- Display documents and their relevant information.
- Provide user interaction for search, organization, sharing, and viewing.

Technology: React Native, Expo, Expo Router, TypeScript.
Status: Implemented.

3.5.2 Module 2: Document Management Module

Purpose:
Manage the user's documents and provide access to document-related operations.
Main responsibilities:

- Retrieve documents.
- Display document information.
- Support document access and download.
- Support document viewing.
- Apply access restrictions before protected operations.

The repository identifies document management as responsible for uploading, listing, sharing, downloading, and verifying documents.

3.5.3 Module 3: Role-Based Access Control Module

Purpose:
Control document access according to the authenticated user's permissions and organizational role.
Main responsibilities:

- Validate the user's authentication token.
- Check authorization before protected operations.
- Apply organization-level access rules.
- Restrict operations to authorized users or members.

The project uses authentication middleware and organization/membership checks for authorization.

3.5.4 Module 4: Document Search and Organization Module

Purpose:
Make document retrieval and organization more efficient.
Main responsibilities:

- Retrieve documents associated with the authorized user or organization.
- Support document searching/retrieval.
- Group and organize documents.
- Provide folder/file organization functionality.

Document search is identified as implemented, while file/folder management is currently partially implemented.

3.5.5 Module 5: Controlled Document Viewing Module

Purpose:
Provide authorized users with controlled access to their documents through the application.
Main responsibilities:

- Verify user access before document retrieval.
- Retrieve the required document data.
- Provide document viewing through the application.
- Prevent unauthorized document access.

Document viewing and download functionality are identified in the existing implementation.

3.6 Non-Functional Requirements

3.6.1 Security

Security is an important requirement because the application manages potentially sensitive documents. Authentication and authorization mechanisms are used to protect application resources. The existing system includes JWT validation, access-control middleware, and encrypted document storage mechanisms.

3.6.2 Usability

The mobile application should provide a simple and accessible interface for managing documents. Frequently required operations such as document retrieval, organization, sharing, and viewing should be accessible without requiring users to interact directly with the underlying backend infrastructure.

3.6.3 Reliability

The mobile application should correctly communicate with the backend and handle API responses and access-control failures appropriately. Document operations should only be performed after the required authentication and authorization checks.

3.6.4 Performance

The application should provide responsive document retrieval and navigation. However, the project analysis does not contain formal benchmark measurements or performance test results at the current stage. Therefore, no specific performance values are claimed in this mid-term report.

3.6.5 Maintainability

The mobile application is maintained as a separate client while reusing the existing backend services. This separation allows mobile-specific screens and functionality to be developed without unnecessarily duplicating backend business logic.

4 Software Design

The software design of the Secure Document Vault defines the overall structure, components, data flow, interactions, and deployment of the system. The design focuses on the integration of the web and mobile applications with the backend services and the supporting database, decentralized storage, and blockchain infrastructure.

The Major Project extends the existing Secure Document Vault by providing a mobile application and strengthening document management, access control, organization, and controlled document access. The design therefore considers the interaction between users, applications, backend services, and storage components while supporting secure document handling and role-based access.

The software design is represented using the following diagrams:

- System Architecture
- System Use Case Diagram
- DFD Level 0 (Context Diagram)
- DFD Level 1
- Sequence Diagram – Public Dual-Check Verification
- Activity Diagram – Controlled Document Access & Decryption
- Class / Entity Diagram
- Deployment Diagram

4.1 System Architecture

The system architecture represents the overall structure of the Secure Document Vault and shows how users interact with the mobile and web applications and how these applications communicate with the backend services. The architecture consists of the **Mobile Application**, **Web Application**, **Backend API**, **MongoDB Database**, **IPFS Storage**, and **Ethereum Sepolia Blockchain**.

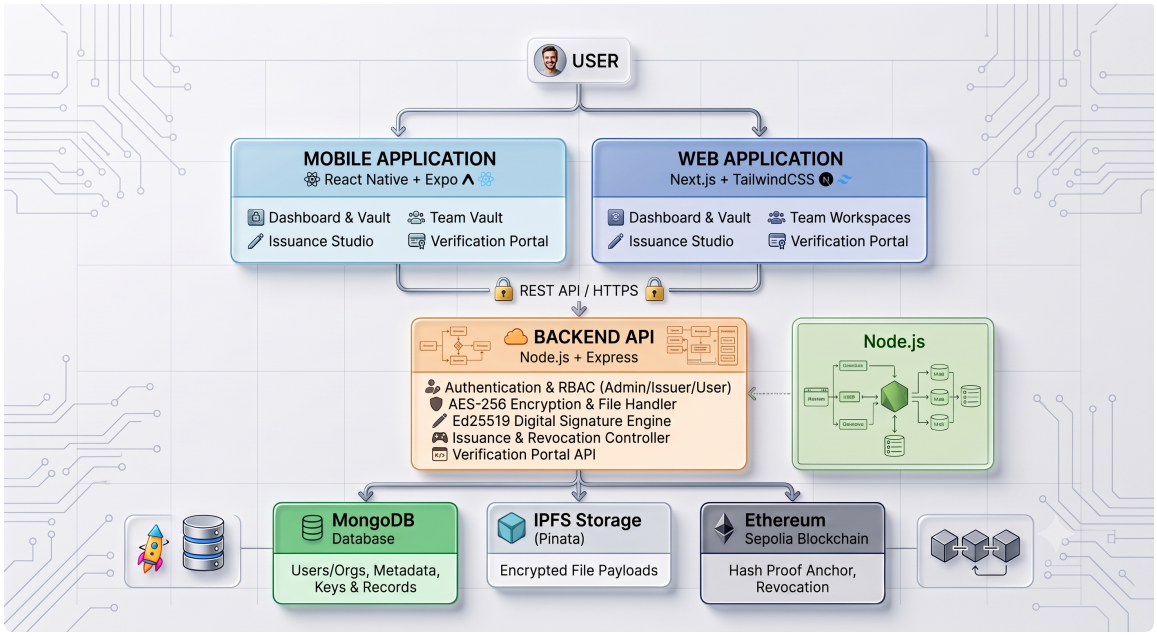

Figure 1: System Architecture of the Secure Document Vault

The system begins with the **User**, who can access the platform through either the mobile application or the web application. The mobile application is developed using **React Native with Expo**, while the web application is developed using **Next.js with TailwindCSS**. Both applications provide document-management and organizational features such as the dashboard and vault, organization directory, and verification functionality. The mobile and web applications also provide additional system-specific features such as team vaults, team workspaces, and issuance functionality.

Both applications communicate with the backend through **REST APIs over HTTPS**. The backend is implemented using **Node.js and Express** and acts as the central processing layer of the system. It manages authentication and role-based access control for **Admin, Issuer, and User** roles. It also handles file processing and **AES-256 encryption**, digital signatures using the **Ed25519** signature engine, issuance and revocation operations, and APIs required by the verification portal.

The backend communicates with three major storage and verification components. **MongoDB** is used to store users, organizations, metadata, keys, and other system records. **IPFS Storage through Pinata** is used to store encrypted file payloads. The **Ethereum Sepolia Blockchain** provides a blockchain-based integrity layer by maintaining hash proof anchors and revocation information.

This architecture separates the user interfaces from the backend processing and storage layers. It allows the mobile application developed as part of the Major Project to use the existing backend infrastructure while providing mobile users with secure document-management, access-control, search, organization, and controlled document-access functionality.

4.2 System Use Case Diagram

The System Use Case Diagram illustrates the major interactions between different types of users and the Secure Document Vault. It identifies four primary actors: **User, Authorized Issuer, Organization Admin, and Public Verifier**. Each actor has a different set of operations based on their responsibilities and access privileges.

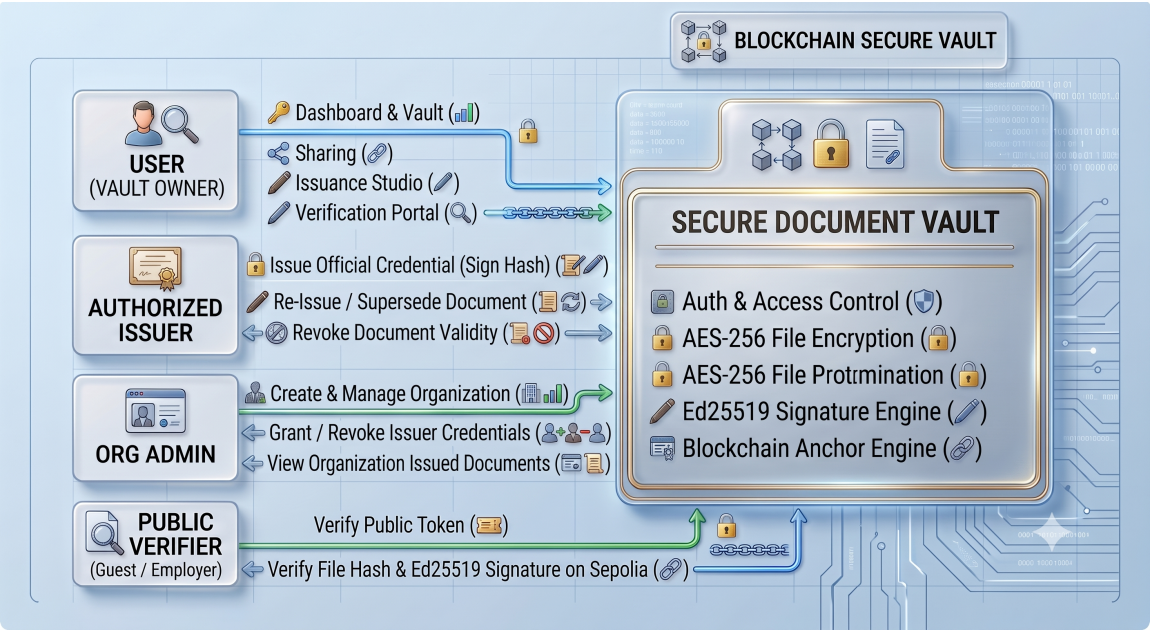

Figure 2: System Use Case Diagram

The **User** is the primary system user and can register and authenticate, upload and encrypt personal files, view documents stored in the personal vault, share documents, and view received

credentials. These operations represent the basic document-management and credential-related functionality available to users.

The **Authorized Issuer** is responsible for issuing and managing official credentials. The issuer can issue an official credential by signing the document hash, re-issue or supersede an existing document, and revoke the validity of a document when required.

The **Organization Admin** manages organization-level operations. The administrator can create and manage an organization, grant or revoke issuer credentials, and view documents issued by the organization.

The **Public Verifier**, which may represent a guest or employer, provides an external verification role. The verifier can verify a public token and verify the file hash and **Ed25519 digital signature** against the information anchored on the **Ethereum Sepolia blockchain**.

The use case diagram therefore defines the responsibilities of each actor and shows how access to system functionality is separated according to user roles. This role separation also supports the **role-based access control** objective of the Major Project.

4.3 DFD Level 0 (Context Diagram)

The Level 0 Data Flow Diagram (DFD), also called the Context Diagram, provides a high-level representation of the data exchanged between the **Blockchain Secure Vault System** and its external actors and storage components. The system is represented as a single central process consisting of the **Web Application, Mobile Application, and Backend**.

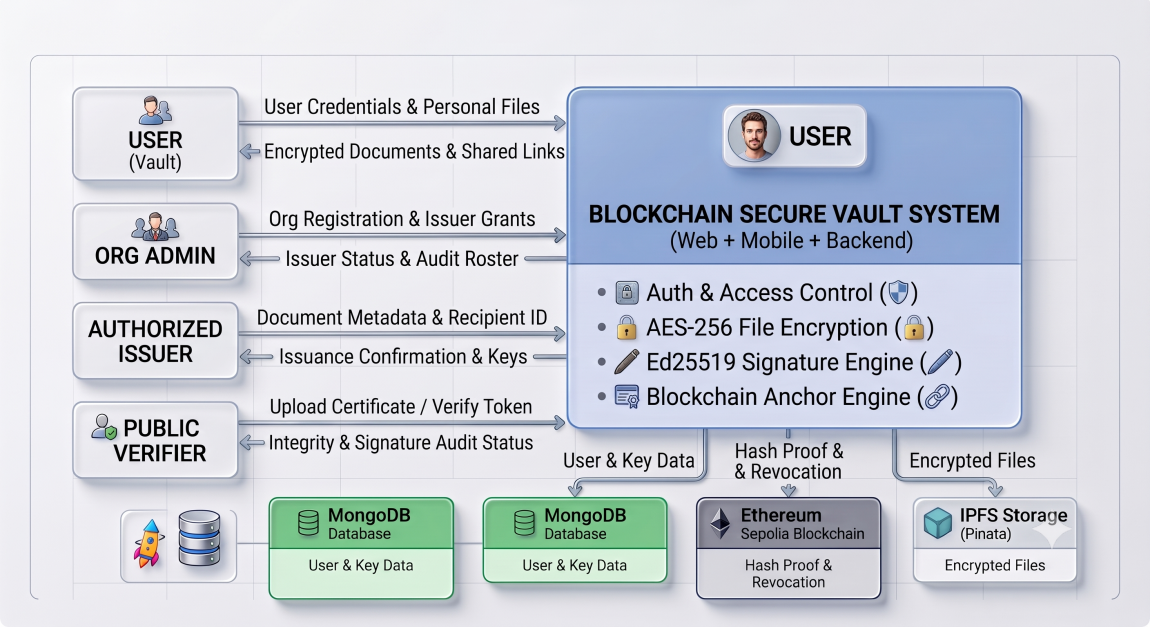

Figure 3: DFD Level 0 (Context Diagram)

The **User (Vault)** interacts with the system by providing user credentials and personal files. In return, the system provides encrypted documents and shared links. This interaction represents the core document-management functionality of the system.

The **Organization Admin** provides organization registration information and issuer grants to the system. The system returns issuer status and the organization audit roster. This interaction represents organization-level administration and access management.

The **Authorized Issuer** provides document metadata and recipient information to the system. The system responds with issuance confirmation and relevant keys. This supports the controlled issuance and management of official documents.

The **Public Verifier** provides a certificate or verification token to the system for verification. The system returns the integrity and signature audit status, allowing the verifier to determine whether the submitted document information is valid.

The central system also communicates with three supporting storage and verification components. **MongoDB** stores user and key data, **Ethereum Sepolia Blockchain** stores hash proofs and revocation information, and **IPFS Storage through Pinata** stores encrypted files.

The DFD therefore provides a high-level view of the major data flows without describing the internal processing steps of the system. It establishes how users and external actors interact with the system and how the system exchanges information with its persistent storage and blockchain components.

4.4 DFD Level 1

The Level 1 Data Flow Diagram provides a more detailed representation of the internal processing of the Blockchain Secure Vault System. It decomposes the system into five major processes: **Authentication and RBAC Control, Personal Vault and AES-256 Encryption, Organization and Issuer Management, Ed25519 Digital Signature and Issuance, and Dual-Check Public Verification Portal**.

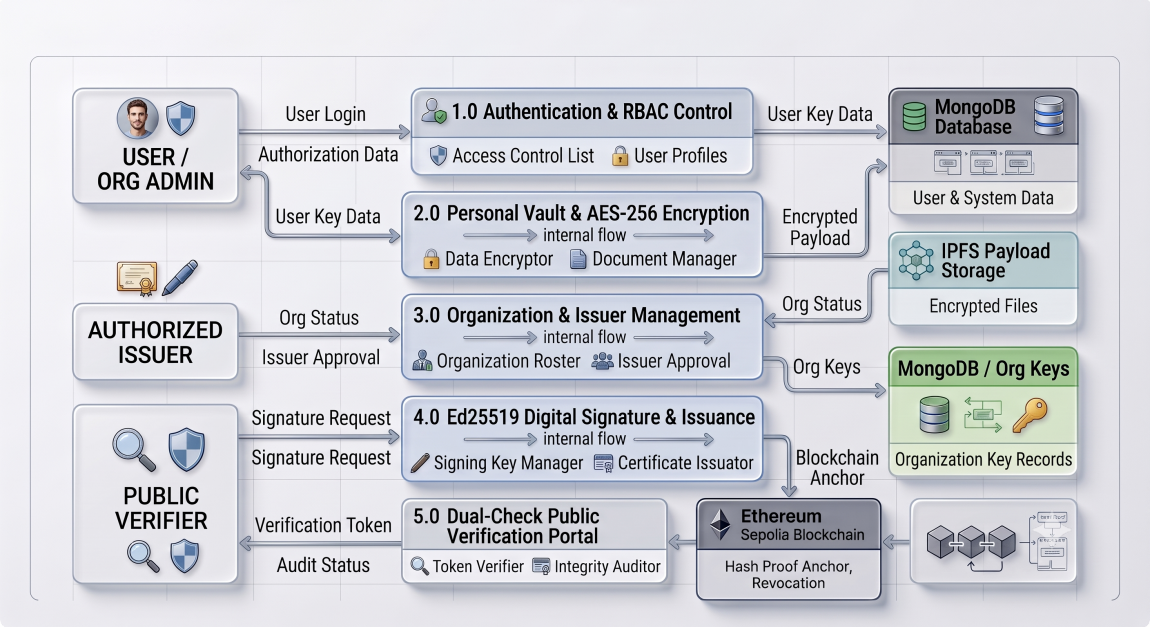

Figure 4: DFD Level 1 of the Blockchain Secure Vault System

The first process, **1.0 Authentication & RBAC Control**, receives authentication and access requests from the User or Organization Admin. It communicates with the **MongoDB Database** to retrieve and maintain the information required for authentication and role-based access control. This process ensures that users can access only the functions permitted by their assigned roles.

The second process, **2.0 Personal Vault & AES-256 Encryption**, handles personal document management and file protection. After authentication and authorization, documents are processed through the personal vault and protected using **AES-256 encryption**. The encrypted payload is then stored in **IPFS Payload Storage**.

The third process, **3.0 Organization & Issuer Management**, handles organization-level operations and issuer management. The Authorized Issuer interacts with this process, while organization-related keys and information are maintained through **MongoDB / Organization Keys**.

The fourth process, **4.0 Ed25519 Digital Signature & Issuance**, handles digital signing and document issuance. It uses the **Ed25519 digital signature mechanism** and communicates with the **Ethereum Sepolia Blockchain** for blockchain-based anchoring and related issuance information.

The fifth process, **5.0 Dual-Check Public Verification Portal**, provides document verification functionality for the Public Verifier. The process receives verification requests and uses information from the Ethereum Sepolia Blockchain to perform the required verification checks before returning the verification result.

Overall, the Level 1 DFD shows how authentication and access control, encrypted document storage, organization management, digital signing, issuance, and public verification are connected within the system. It provides more detail than the Level 0 context diagram while still maintaining a high-level view of the major system processes.

4.5 Sequence Diagram – Public Dual-Check Verification

The sequence diagram illustrates the step-by-step process followed when a **Public Verifier** checks the authenticity of a document through the public verification portal. The verification process involves the **Verifier, Public Portal, Backend API, and Ethereum Sepolia Blockchain**.

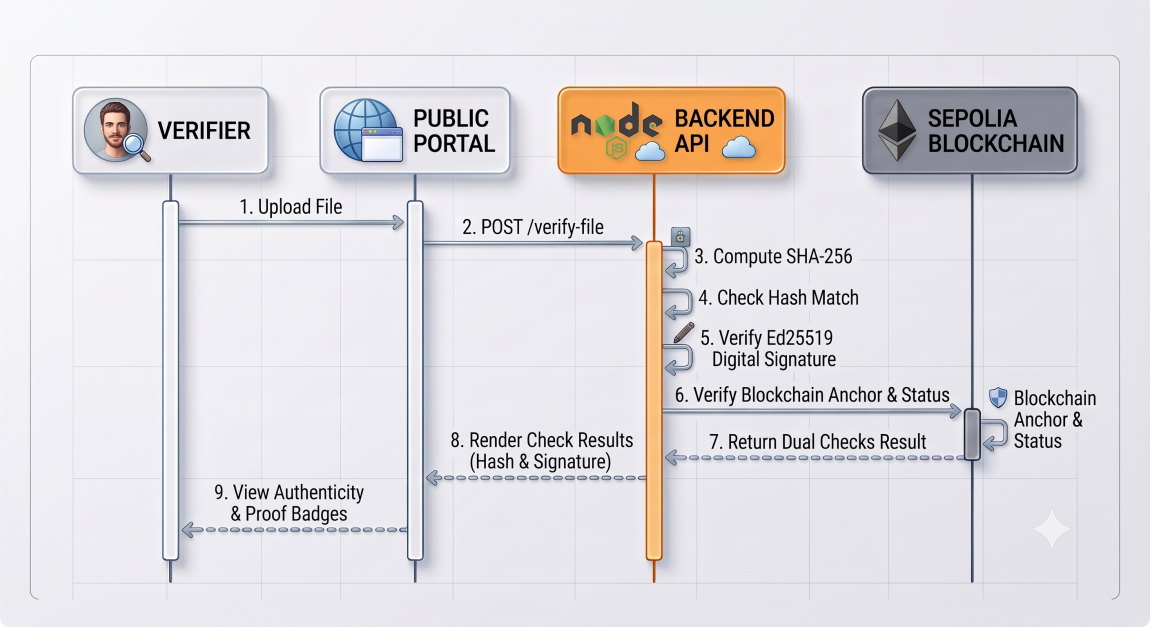

Figure 5: Sequence Diagram for Public Dual-Check Verification

The process begins when the **Verifier uploads a file** through the Public Portal. The portal sends a `POST /verify-file` request to the Backend API to initiate the verification process.

The Backend API then computes the **SHA-256 hash** of the uploaded file and checks whether the calculated hash matches the expected document hash. It also verifies the **Ed25519 digital signature** associated with the document.

After performing the file-level checks, the Backend API communicates with the **Ethereum Sepolia Blockchain** to verify the blockchain anchor and the current status of the document. The blockchain response is returned to the backend, which combines the results of the hash, digital signature, and blockchain checks.

The Backend API then returns the **dual-check verification result** to the Public Portal. The portal renders the verification results, including the hash and signature checks, for the verifier. Finally, the verifier can view the document's **authenticity status and proof badges**.

This sequence demonstrates how the verification process combines file-level integrity checking with digital-signature and blockchain verification to provide an authenticity result to an external verifier.

4.6 Activity Diagram – Controlled Document Access & Decryption

The activity diagram represents the workflow followed when an authenticated user attempts to access a document from the personal vault. It focuses on the authorization check performed before the encrypted document is retrieved and decrypted.

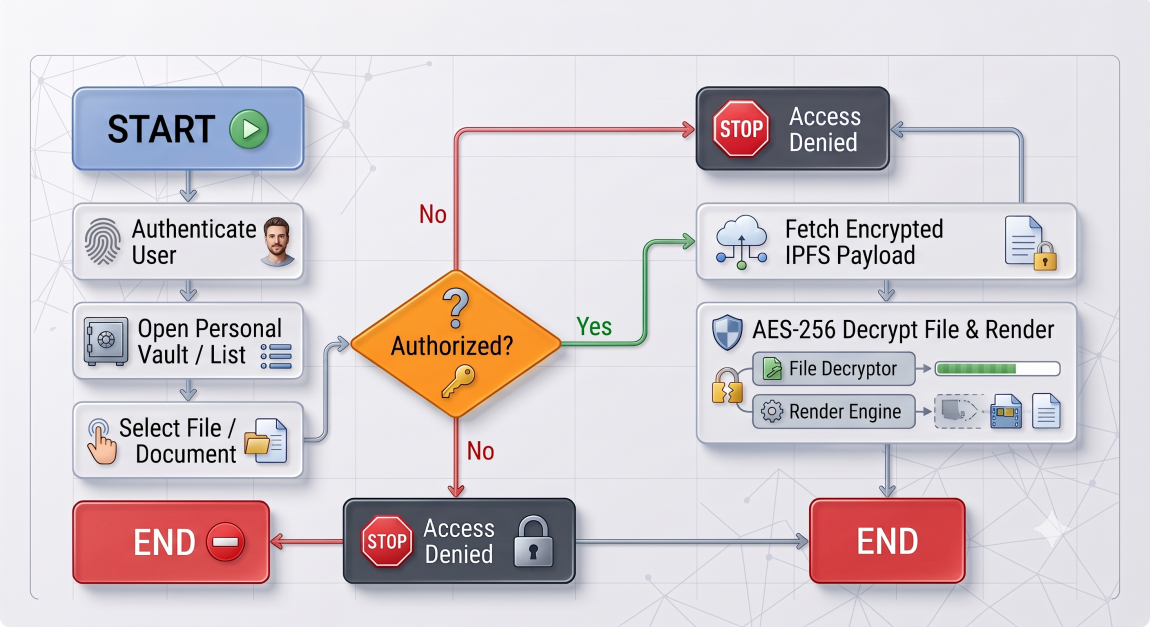

Figure 6: Activity Diagram for Controlled Document Access and Decryption

The process begins when the user **authenticates** with the system. After successful authentication, the user opens the **Personal Vault** and views the available documents. The user then selects a file or document for access.

The system performs an **authorization check** to determine whether the user is permitted to access the selected document. If the user is not authorized, the system generates an **Access Denied** response and terminates the access workflow.

If the user is authorized, the system retrieves the **encrypted IPFS payload** associated with the selected document. The encrypted file is then processed using **AES-256 decryption**, after which the decrypted file is rendered for the authorized user.

The workflow ensures that document access is controlled before the encrypted file is retrieved and displayed. This supports secure document access by combining authorization checks with encrypted document storage and controlled decryption.

4.7 Class / Entity Diagram

The Class / Entity Diagram represents the major data entities involved in the Secure Document Vault and the relationships between them. The diagram focuses on users, organizations, issuer authorization, documents, issuance records, and document-sharing links.

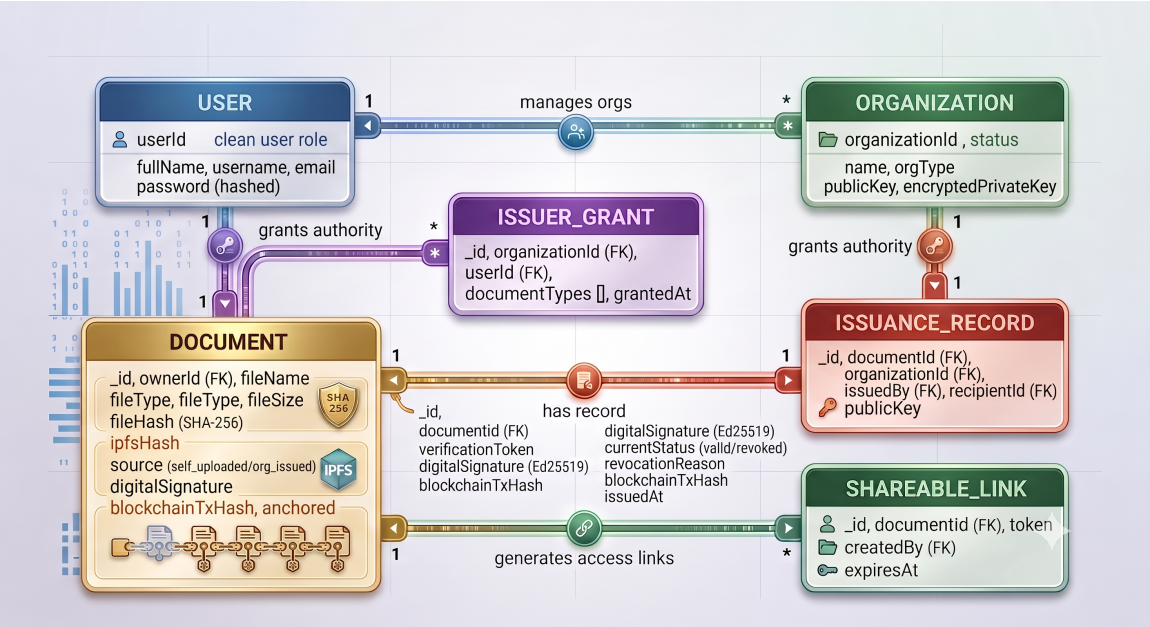

Figure 7: Class / Entity Diagram of the Secure Document Vault

The **User** entity stores information related to system users, including their identity details, hashed password, and assigned role. A user can manage organizations and can be associated with organization-level authority.

The **Organization** entity represents an organization registered within the system. It contains organization information, status, and cryptographic key information. Organizations are connected with users through organizational management and issuer authority.

The **Issuer Grant** entity represents the authority granted to a user to perform issuer-related operations within an organization. It records the organization, user, permitted document types, and the time at which the authority was granted.

The **Document** entity represents files managed by the system. It stores information such as the owner, file name, file type, file size, **SHA-256 file hash**, IPFS hash, document source, digital signature, and blockchain transaction information. These attributes support document identification, integrity verification, and secure storage.

The **Issuance Record** entity maintains information about documents issued by an organization. It contains the document and organization references, issuer and recipient information, verification token, **Ed25519 digital signature**, current document status, revocation information, blockchain transaction hash, and issuance timestamp.

The **Shareable Link** entity represents an access link generated for sharing a document. It maintains the associated document, access token, creator information, and expiration time. This allows document access to be shared through controlled and potentially time-limited links.

The relationships shown in the diagram establish how users, organizations, documents, issuer permissions, issuance records, and shareable links are connected. These relationships provide the underlying data structure required for document ownership, organizational issuance, authorization, and controlled document sharing.

4.8 Deployment Diagram

The Deployment Diagram represents the physical and runtime arrangement of the major components of the Secure Document Vault. It shows how the **mobile client, web client, backend server, database, decentralized storage, and blockchain network** are connected during system operation.

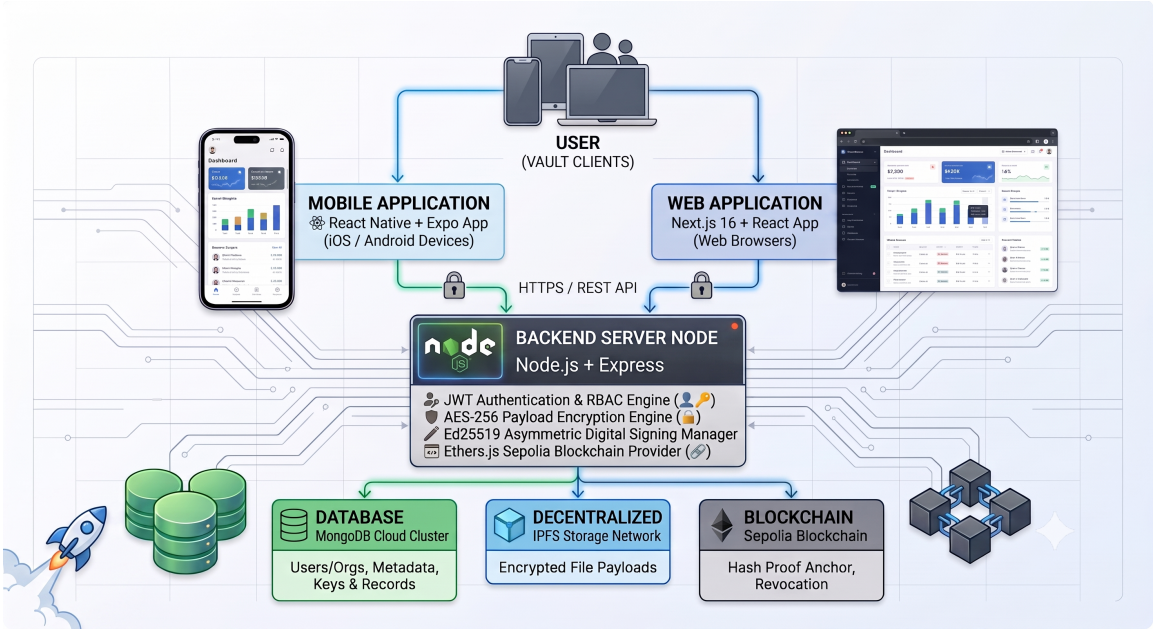

Figure 8: Deployment Diagram of the Secure Document Vault

The **Mobile Client Node** represents mobile devices running the application developed using **React Native and Expo**. The application can operate on iOS and Android devices and provides mobile access to the document vault and other system functionality.

The **Web Client Node** represents web browsers running the **Next.js and React** application. Both the mobile and web clients communicate with the backend server through **HTTPS using REST APIs**.

The **Backend Server Node** hosts the Node.js and Express runtime and contains the main server-side processing components. These include the **JWT Authentication and RBAC Engine**, **AES-256 Payload Encryption Engine**, **Ed25519 Asymmetric Digital Signing Manager**, and the **Ethers.js Sepolia Blockchain Provider**.

The backend server communicates with three external infrastructure nodes. The **Database Node** contains the MongoDB Cloud Cluster for persistent application data. The **Decentralized Node** represents the IPFS storage network used for decentralized file storage. The **Blockchain Node** represents the Ethereum Sepolia network used for blockchain-based document verification and related blockchain operations.

The deployment structure separates client applications, backend processing, persistent database storage, decentralized file storage, and blockchain services. This separation allows the mobile and web applications to access the same backend services while maintaining dedicated infrastructure for data storage, encrypted file storage, and blockchain-based verification.

4.9 Design Summary

The software design provides a structured representation of the Secure Document Vault from multiple perspectives. The **System Architecture** describes the major application, backend, storage, and blockchain components and their communication. The **Use Case Diagram** defines the responsibilities and interactions of the User, Authorized Issuer, Organization Admin, and Public Verifier.

The **DFD Level 0** provides a high-level representation of the data exchanged between the system and its external actors, while the **DFD Level 1** further decomposes the system into authentication, document encryption, organization management, digital signing and issuance, and public verification processes.

The **Sequence Diagram** demonstrates the public document verification workflow, including SHA-256 hash calculation, Ed25519 signature verification, and blockchain-anchor verification. The **Activity Diagram** demonstrates controlled document access, where authorization is checked before encrypted content is retrieved and decrypted.

The **Class / Entity Diagram** represents the main data entities and their relationships, including users, organizations, issuer grants, documents, issuance records, and shareable links. Finally, the **Deployment Diagram** shows how the mobile and web clients communicate with the backend server and how the backend interacts with MongoDB, IPFS, and Ethereum Sepolia.

Together, these diagrams provide a comprehensive design view of the system while supporting the Major Project objectives related to **secure mobile document management, role-based access control, document organization and sharing, and controlled document access and verification**.

5 Testing Module

Testing is an important part of the Major Project because the mobile application handles document management and controlled access to potentially sensitive information. The testing process is used to verify that the implemented functionality works according to the specified requirements and that unauthorized users cannot access protected operations.

5.1 Testing Objectives

The main objectives of testing are:

- To verify that the mobile application functions correctly.
- To verify communication between the mobile application and backend APIs.
- To verify that authenticated users can access authorized functionality.
- To verify role-based access restrictions.
- To verify document retrieval and search functionality.
- To verify document viewing and sharing operations.
- To identify errors in the application and correct them during development.
- To ensure that partially developed functionality is clearly identified rather than treated as complete.

5.2 Testing Techniques

The following testing techniques are applicable to the Major Project.

5.2.1 Functional Testing

Functional testing verifies whether individual features perform their intended operations.

The major functionality tested includes:

- User authentication
- Mobile navigation
- Document retrieval
- Document search
- Document viewing
- Document sharing
- Role-based access
- API communication
- Access-denied scenarios

5.2.2 Integration Testing

Integration testing verifies the interaction between different components of the system.

For the Major Project, the primary integration points are:

- **Mobile Application → Backend API**
- **Backend API → Database**
- **Backend API → Existing Document Services**

The purpose is to ensure that requests generated by the mobile application are correctly processed by the backend and that appropriate responses are returned.

5.2.3 Access-Control Testing

Access-control testing is particularly important because one of the Major Project objectives is Role-Based Access Control.

The testing verifies that:

- An authenticated user can access permitted operations.
- A user without the required permission cannot perform restricted operations.
- Organization or role-based restrictions are correctly applied.
- Document sharing is limited according to authorization rules.
- Protected API requests are rejected when authentication or authorization requirements are not satisfied.

The existing project contains JWT validation and organization/membership-based authorization mechanisms.

5.2.4 User Interface Testing

The mobile interface is tested to verify:

- Correct navigation between screens.
- Correct display of document information.
- Proper response to user actions.
- Correct handling of loading states.
- Appropriate display of errors and access-denied messages.
- Usability of document-management operations.

5.3 Test Cases

The following test cases represent the main functionality relevant to the Major Project.

Table 4: Test Cases for the Major Project

Test Case ID	Test Scenario	Input / Action	Expected Result
TC-01	User Login	Enter valid login credentials	User is authenticated and allowed to access the application
TC-02	Invalid Login	Enter invalid credentials	Login request is rejected and an error is displayed
TC-03	Document Retrieval	Open document section as an authorized user	Authorized documents are displayed
TC-04	Document Search	Search using a document-related query	Relevant documents are retrieved
TC-05	Unauthorized Document Access	Attempt to access a restricted document	Access is denied
TC-06	Role-Based Access	Perform an operation with an unauthorized role	Operation is rejected
TC-07	Document Sharing	Share a document with an authorized user	Sharing request is processed according to permissions
TC-08	Document Viewing	Open an accessible document	Document is displayed through the permitted viewing mechanism
TC-09	API Communication	Perform a document operation from mobile app	Request reaches backend and appropriate response is returned
TC-10	Invalid API Request	Send an invalid or unauthorized request	Backend rejects the request appropriately
TC-11	Navigation	Move between relevant mobile screens	Correct destination screen is displayed
TC-12	Folder/File Organization	Create or use available organization functionality	Documents are organized according to the implemented functionality

5.4 Security Testing

Security testing focuses on preventing unauthorized access to documents and protected application functionality.

The following scenarios are considered:

1. **Authentication Validation**

   Requests to protected resources should require valid authentication.

2. **Authorization Validation**

Authentication alone should not provide access to operations that require additional permissions.

3. **Role Validation**

   User roles and organizational membership should be considered when processing restricted operations.

4. **Unauthorized Access**

   Attempts to access documents without the required permissions should be rejected.

5. **Session Security**

   Authentication information used by the mobile application should be handled using appropriate secure storage mechanisms.

The existing system includes JWT authentication, authorization middleware, token validation, and access-control checks.

5.5 Expected Testing Outcome

Successful testing should demonstrate that:

- The mobile application communicates correctly with the backend.
- Authorized users can access their permitted documents.
- Unauthorized users cannot perform restricted document operations.
- Role-based permissions are enforced.
- Documents can be searched and retrieved according to the implemented functionality.
- Documents can be viewed through the permitted application workflow.
- The application handles invalid requests and errors appropriately.
- The remaining folder-based organization functionality can be verified after its implementation is completed.

Thus, the testing process provides a systematic method for validating the Major Project objectives while identifying functionality that still requires development before the final project evaluation.

6 Performance of Project Developed So Far

The performance evaluation of the Major Project focuses on the functionality developed up to the mid-term stage. Since the project is an extension of the existing Secure Document Vault, the evaluation primarily considers the performance of the newly developed mobile application, its communication with the backend, document-management operations, and access-control mechanisms.

At the current stage, the project provides an Expo/React Native mobile application that communicates with the existing backend through REST APIs. The repository analysis confirms that the mobile application and core document-management and access-control functionality are implemented, while file/folder management remains partially implemented.

6.1 Mobile Application Performance

The mobile application provides a mobile interface for accessing the Secure Document Vault. The application communicates with the backend instead of directly accessing the database or storage services.

The separation between the mobile application and backend provides the following advantages:

- The mobile application remains focused on user interaction and presentation.
- Existing backend business logic can be reused.
- Database operations remain isolated from the mobile client.
- Authentication and authorization can be handled centrally.
- Changes to the backend can be utilized by both web and mobile clients.

The project architecture confirms that the web and mobile applications communicate with the same backend API.

6.2 Document Management Performance

The implemented document-management functionality allows users to retrieve and interact with documents through the application.

The existing system supports document listing, sharing, downloading, viewing, and verification operations. Document search/retrieval according to user or organization context is also identified as implemented.

The performance of these operations depends on factors such as:

- Network connectivity.
- Backend response time.
- Database query processing.
- Document size.
- Storage retrieval time.
- Mobile device resources.

No formal benchmark data for response time or throughput is currently available in the repository. Therefore, numerical performance claims are not included in this mid-term evaluation.

6.3 Role-Based Access Control Performance

Role-Based Access Control provides an additional authorization layer for protected document operations.

When a user requests a protected operation, the request is processed through the authentication and authorization mechanisms before the operation is performed. This prevents unauthorized users from directly accessing restricted resources.

The project contains JWT validation, organization-level checks, and membership-based authorization. Role-based access control is identified as an implemented requirement.

This architecture allows access decisions to be handled by the backend rather than relying only on restrictions implemented in the mobile interface.

7 Output Screens

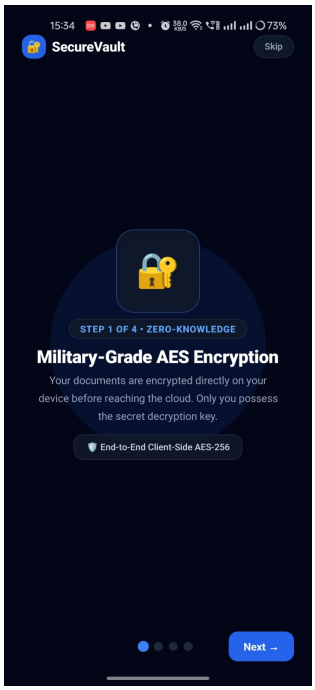

Figure 9: Secure Document Vault Home Page – View 1

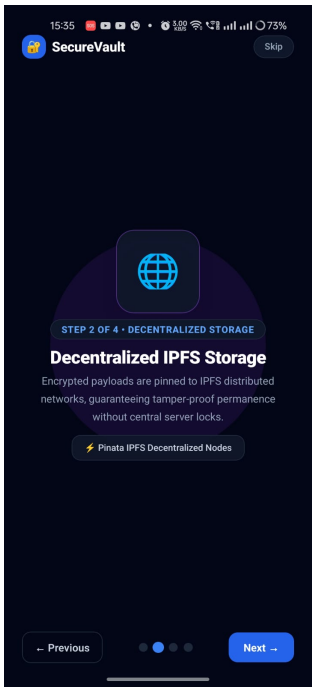

Figure 10: Secure Document Vault Home Page – View 2

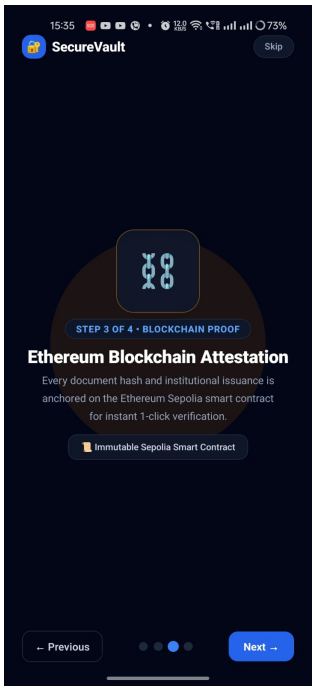

Figure 11: Secure Document Vault Home Page – View 3

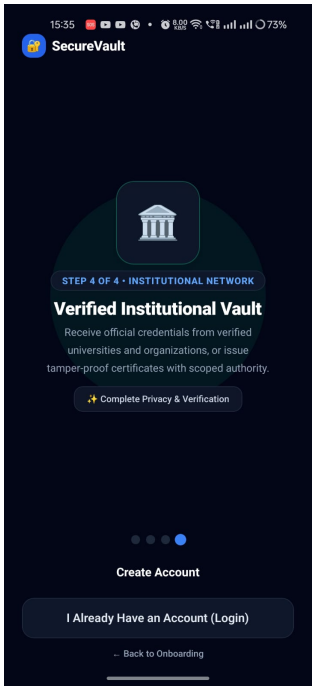

Figure 12: Secure Document Vault Home Page – View 4

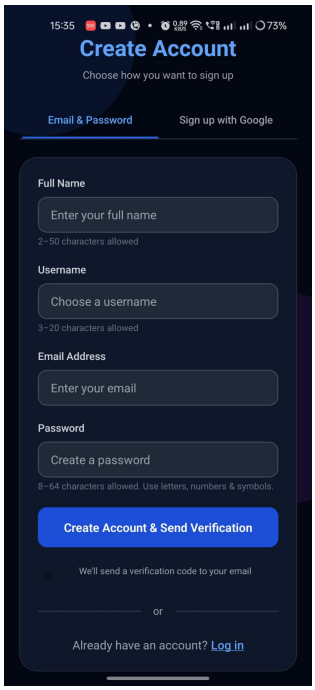

Figure 13: User Registration Using Email and Password

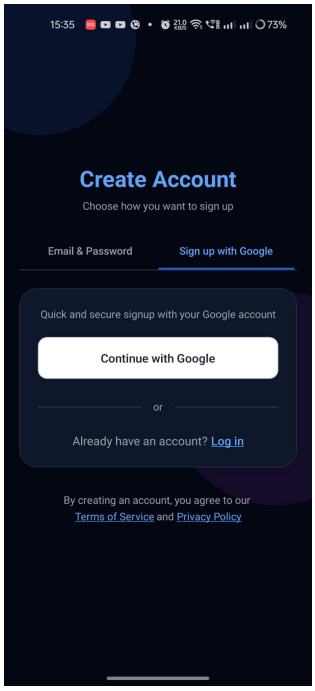

Figure 14: User Registration Using Google Authentication

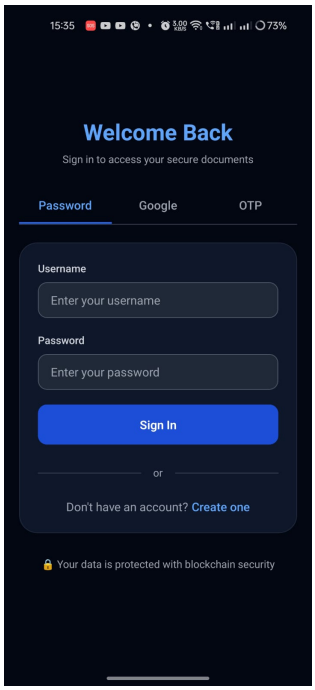

Figure 15: User Sign In Using Email and Password

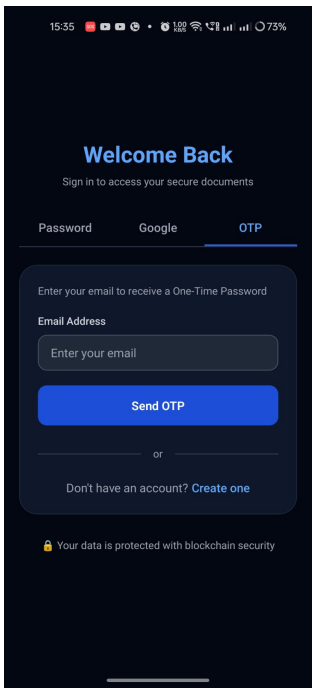

Figure 16: User Registration Using OTP Verification

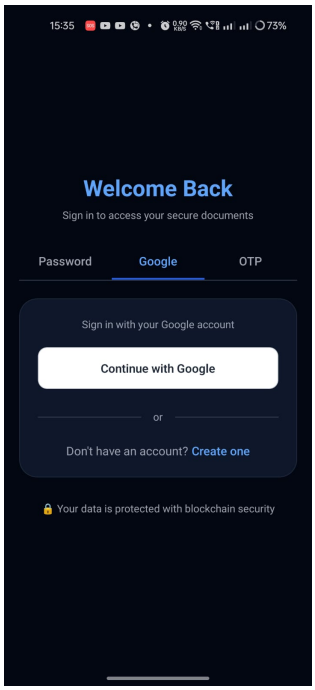

Figure 17: User Sign In Using Google Authentication

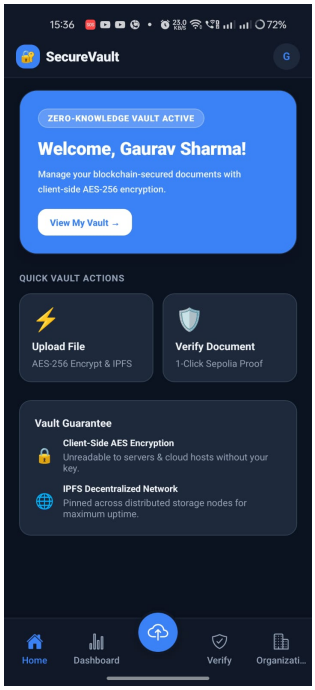

Figure 18: Secure Document Vault Home Page After User Authentication

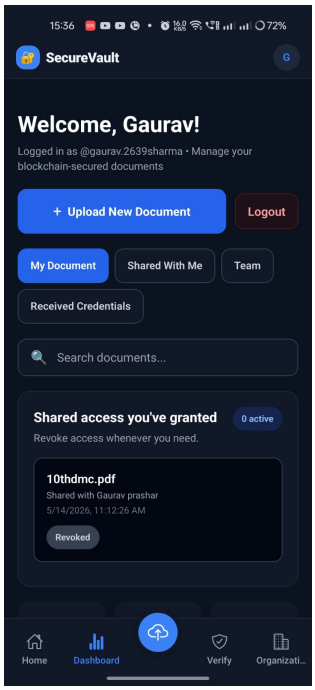

Figure 19: Secure Document Vault Dashboard

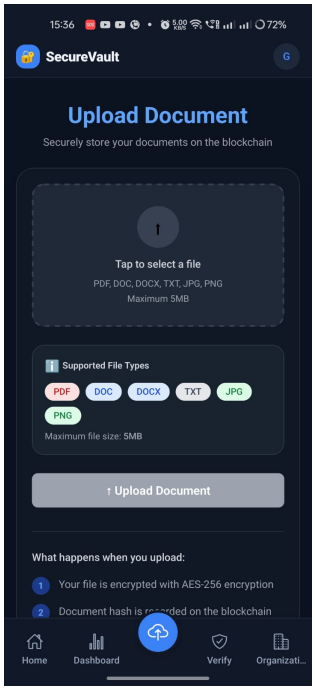

Figure 20: Document Upload Interface

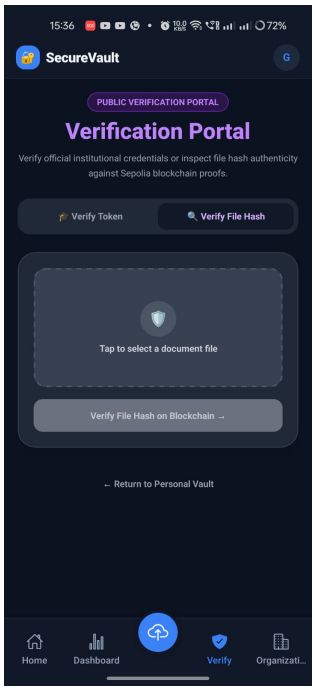

Figure 21: Document Verification Interface

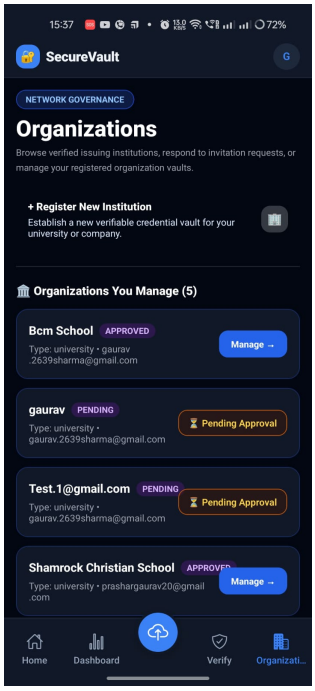

Figure 22: Organization Dashboard

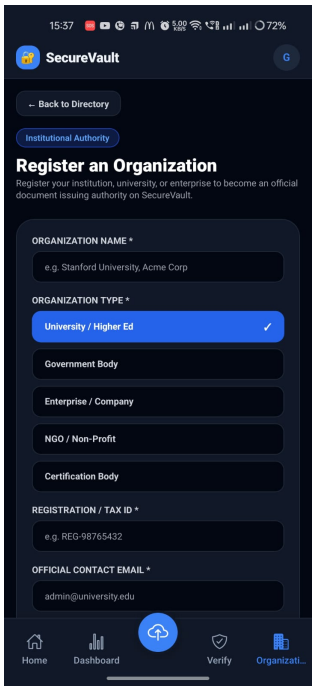

Figure 23: Organization Registration Interface

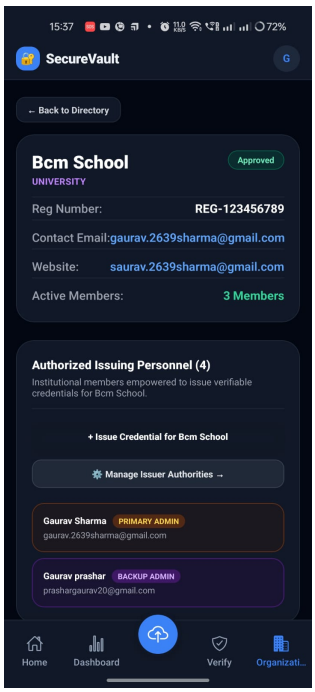

Figure 24: Organization Management Interface

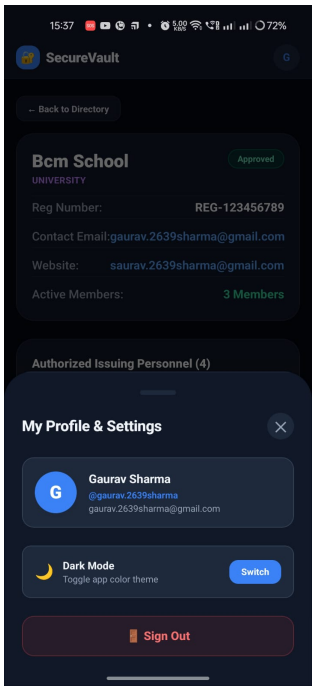

Figure 25: User Profile Information and Logout Interface

References

[1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed. London, UK: Pearson, 2017.

[2] National Institute of Standards and Technology, "Fips 197: Advanced encryption standard (aes)," 2001, federal Information Processing Standards Publication 197. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/197/final